

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 10
IMPLEMENTASI JAVA COLLECTIONS FRAMEWORK
(SET, LIST, DAN MAP)



Disusun Oleh:

Glory Hardy Febriando Sianturi

(105223013)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Program simulasi "Sistem Manajemen Gudang" ini adalah sebuah arsitektur *backend* pengelolaan data inventaris berbasis *Command Line Interface* (CLI) yang dirancang untuk menerapkan konsep dasar Pemrograman Berorientasi Objek (PBO) murni yang dikombinasikan dengan keluwesan struktur data.

Tujuan utama dari program ini adalah mengimplementasikan *Java Collections Framework* secara komprehensif, khususnya penggunaan antarmuka `Map`, `Set`, dan `List`. Penggunaan *collections* ini menggantikan struktur *array* statis tradisional untuk memberikan fleksibilitas alokasi memori dinamis dan efisiensi dalam manipulasi data.

Tantangan utama dalam pengembangan program ini adalah membangun logika penyimpanan data yang terintegrasi: memastikan pencarian barang berjalan sangat cepat melalui pemetaan kunci unik (ID Barang), menolak secara otomatis duplikasi data kategori yang diinput ke dalam sistem, serta mempertahankan urutan kronologis yang akurat untuk setiap riwayat transaksi (barang masuk, tambah stok, dan kurangi stok).

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	<i>idBarang</i>	String	Menyimpan identitas unik untuk setiap barang sebagai <i>Primary Key</i> .
2	namaBarang	String	Menyimpan deskripsi nama barang di dalam entitas tunggal.
3	kategori	String	Menyimpan klasifikasi jenis barang.
4	stok	int	Menyimpan jumlah ketersediaan unit barang yang bisa dimanipulasi.
5	databaseBarang	Map<String, Barang>	(<i>Map</i>) Penyimpan data utama. Menggunakan <i>idBarang</i> sebagai <i>Key</i> untuk pencarian instan ($O(1)$).
6	kategoriUnik	Set	(<i>Set</i>) Mengumpulkan nama kategori dan secara otomatis menolak nilai ganda/duplikat.
7	riwayat	List	(<i>List</i>) Mencatat string log setiap aktivitas ke dalam memori secara berurutan.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	Barang()	<i>Constructor</i>	Menginisialisasi atribut dasar (idBarang, namaBarang, kategori, stok) saat objek dicetak.
2	tambahBarangBaru()	<i>Procedural</i>	Memasukkan objek Barang baru ke dalam databaseBarang (Map) dan mencatat kategorinya ke kategoriUnik (Set).
3	tambahStok()	<i>Procedural</i>	Melakukan pencarian instan via <i>Key</i> dan menambahkan nilai integer pada atribut stok objek tersebut.
4	kurangiStok()	<i>Procedural</i>	Mengeksekusi pengurangan ketersediaan dengan logika validasi: menolak transaksi jika sisa stok tidak mencukupi.
5	cetakLaporan()	<i>Procedural</i>	Menampilkan hasil iterasi iteratif dari ketiga <i>Collections</i> (Map, Set, List) ke layar konsol terminal.

IV. Dokumentasi dan Pembahasan Code

4.1. Pembuatan Class Model (Barang.java)

Pada bagian ini, sistem mendefinisikan entitas murni yang berfungsi sebagai representasi cetak biru objek barang.

Kode Barang.java:

```
src > src > Barang.java > Barang
Windsurf: Refactor | Explain
1 public class Barang {
2     String idBarang;
3     String namaBarang;
4     String kategori;
5     int stok;
6
7     public Barang(String idBarang, String namaBarang, String kategori, int stok) {
8         this.idBarang = idBarang;
9         this.namaBarang = namaBarang;
10        this.kategori = kategori;
11        this.stok = stok;
12    }
13
14    Windsurf: Refactor | Explain | Generate Javadoc | X
15    @Override
16    public String toString() {
17        return "ID: " + idBarang + " | Nama: " + namaBarang + " | Kategori: " + kategori + " | Stok: " + stok;
18    }
19 }
```

Pembahasan:

Kelas ini tidak memiliki logika bisnis yang rumit; kelas ini hanya bertugas menampung data (POJO - *Plain Old Java Object*). Metode `toString()` di-*override* untuk mempermudah pencetakan objek secara keseluruhan ketika dipanggil oleh sistem *Collections* nantinya.

4.2. Implementasi Java Collections (SistemGudang.java)

Sistem gudang bertindak sebagai *controller* yang mengelola tiga struktur data utama secara dinamis tanpa menggunakan *static keyword*.

Kode SistemGudang.java:

```

src > SistemGudang.java > SistemGudang > tambahStok(String, int)
1 import java.util.*;
2
3 Windsurf Refactor | Explain
4 public class SistemGudang {
5     // Deklarasi Collections sebagai atribut kelas (non-static)
6     Map<String, Barang> databaseBarang = new HashMap<>();
7     Set<String> kategoriUnik = new HashSet<>();
8     List<String> riwayat = new ArrayList<>();
9
10    // Method tambah Barang Baru
11    Windsurf Refactor | Explain | X
12    public void tambahBarangBaru(String id, String nama, String kategori, int stok) {
13        if (!databaseBarang.containsKey(id)) {
14            Barang barangBaru = new Barang(id, nama, kategori, stok);
15            databaseBarang.put(id, barangBaru); // Masuk ke Map
16            kategoriUnik.add(kategori); // Masuk ke Set
17            riwayat.add("Barang Baru Masuk: " + nama + " (" + id + ") ditambah " + stok + " unit");
18        } else {
19            System.out.println("Gagal: Barang dengan ID " + id + " sudah terdaftar!");
20        }
21    }
22
23    // Method tambah stok
24    Windsurf Refactor | Explain | X
25    public void tambahStok(String id, int jumlah) {
26        if (databaseBarang.containsKey(id)) {
27            Barang b = databaseBarang.get(id);
28            b.stok += jumlah;
29            riwayat.add("Update Stok: " + id + " ditambah " + jumlah + " unit. Total: " + b.stok);
30        } else {
31            System.out.println("Gagal Tambah Stok: ID Barang " + id + " tidak ditemukan.");
32        }
33    }
34
35    // Method kurangi stok
36    Windsurf Refactor | Explain | X
37    public void kurangiStok(String id, int jumlah) {
38        if (databaseBarang.containsKey(id)) {
39            Barang b = databaseBarang.get(id);
40            if (b.stok >= jumlah) { // Cek jika stok mencukupi
41                b.stok -= jumlah;
42                riwayat.add("Barang Keluar: " + id + " dikurangi " + jumlah + " unit. Sisa: " + b.stok);
43            } else {
44                // Tolak jika stok kurang
45                System.out.println("[X] DITOLAK: Stok " + b.namaBarang + " (" + id + ") tidak mencukupi! Sisa stok: " + b.stok);
46                riwayat.add("Gagal Barang Keluar: " + id + " - Stok tidak cukup untuk dikurangi " + jumlah);
47            }
48        } else {
49            System.out.println("Gagal Kurangi Stok: ID Barang " + id + " tidak ditemukan.");
50        }
51    }
52
53    // Method cetak laporan akhir
54    Windsurf Refactor | Explain | X
55    public void cetakLaporan() {
56        System.out.println(x: "\n===== LAPORAN SISTEM GUDANG =====");
57
58        System.out.println(x: "\n[1] DAFTAR KATEGORI UNIK (Dari Set):");
59        for (String k : kategoriUnik) {
60            System.out.println("- " + k);
61        }
62
63        System.out.println(x: "\n[2] SISA STOK SEMUA BARANG (Dari Map):");
64        for (Barang b : databaseBarang.values()) {
65            System.out.println("- " + b.toString());
66        }
67
68        System.out.println(x: "\n[3] RIWAYAT TRANSAKSI (Dari List):");
69        for (int i = 0; i < riwayat.size(); i++) {
70            System.out.println((i + 1) + ". " + riwayat.get(i));
71        }
72        System.out.println(x: "=====");
73    }
74 }

```

Pembahasan:

Pengelolaan data didesain sangat terstruktur berdasarkan sifat *Collections*. HashMap dimanfaatkan di dalam metode manipulasi stok untuk mengecek eksistensi barang (menggunakan `.containsKey()`) tanpa harus menggunakan *looping*, sehingga performanya maksimal. HashSet digunakan saat pendaftaran barang untuk mengeliminasi kategori duplikat secara otomatis saat `.add()` dipanggil.

4.3. Simulasi Eksekusi Logika Bisnis (Main.java)

Kelas ini berfungsi sebagai tempat uji coba (*driver class*) yang mengeksekusi operasi secara *hardcode* untuk mendemonstrasikan bahwa sistem berjalan dengan benar.

Kode Main.java:

```

src > Main.java > Main
Windsurf: Refactor | Explain
1 public class Main {
2     Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc | X
3     //instance/objek dari SistemGudang
4     SistemGudang gudang = new SistemGudang();
5
6     System.out.println(x: "=== MEMULAI SIMULASI GUDANG ===\n");
7
8     // 1. Daftarkan minimal 3 barang baru
9     gudang.tambahBarangBaru(id: "B01", nama: "Laptop Asus", kategori: "Elektronik", stok: 10);
10    gudang.tambahBarangBaru(id: "B02", nama: "Meja Belajar", kategori: "Furnitur", stok: 15);
11    gudang.tambahBarangBaru(id: "B03", nama: "Mouse Logitech", kategori: "Elektronik", stok: 20);
12
13    System.out.println(x: "-> 3 Barang berhasil ditambahkan.");
14
15    // 2. Lakukan 1x tambah stok yang berhasil
16    System.out.println(x: "\nMelakukan penambahan stok...");
17    gudang.tambahStok(id: "B01", jumlah: 5);
18
19    // 3. Lakukan 1x kurangi stok yang berhasil
20    System.out.println(x: "\nMelakukan pengurangan stok (berhasil)...");
21    gudang.kurangiStok(id: "B02", jumlah: 5);
22
23    // 4. Lakukan 1x kurangi stok yang GAGAL
24    System.out.println(x: "\nMelakukan pengurangan stok (harus gagal)...");
25    gudang.kurangiStok(id: "B03", jumlah: 50);
26
27    // 5. Panggil metode cetak laporan akhir
28    gudang.cetakLaporan();
29 }
30

```

Pembahasan:

Berdasarkan eksperimen di dalam eksekusi Main.java, logika manajemen gudang terbukti aman. Percobaan manipulasi kurangiStok sebanyak 50 unit pada stok yang hanya berisi 20 unit langsung dicegat oleh pengkondisian *if-else*, dan penolakan tersebut dicatat dengan rapi ke dalam memori berurutan pada kelas List riwayat. Selain itu, meskipun "Elektronik" dimasukkan dua kali, laporan akhir pada struktur Set akan tetap hanya mencetaknya satu kali.

V. Kesimpulan

Berdasarkan hasil perancangan arsitektur Sistem Manajemen Gudang, dapat ditarik kesimpulan sebagai berikut:

- **Efisiensi Pencarian (*Mapping*):** Penerapan antarmuka Map (melalui *HashMap*) memungkinkan program mencari, menambah, dan mengurangi data objek yang spesifik dalam kapasitas besar secara instan dengan bermodalkan *Key* (ID Barang) tunggal.
- **Integritas Data Unik:** Pemanfaatan Set (melalui *HashSet*) terbukti secara logis mengambil alih beban pengecekan data manual (*if-exists*) yang biasa dilakukan *programmer*. Sistem otomatis menolak masuknya entitas kategori yang sudah pernah ada di memori.
- **Pencatatan Kronologis (Sekuensial):** Penggunaan List (melalui *ArrayList*) menjamin semua log aktivitas yang terjadi terekam dengan urutan yang pasti dari awal hingga akhir, yang menjadikannya struktur paling cocok untuk sistem riwayat transaksi.

VI. Daftar Pustaka

- Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education. (Referensi untuk pemahaman tipe data dan operator aritmatika).
- Oracle. (2026). *Java SE Documentation: Class Scanner*. [Online]. Tersedia di: <https://docs.oracle.com/en/java/> (Referensi untuk penggunaan input stream).
- Schildt, H. (2018). *Java: The Complete Reference*. McGraw-Hill Education.